

EJERCICIO 4

Se tienen dos ficheros de acceso secuencial con información referente a los artículos de un almacén:

- Uno llamado "precios.dat", cuyos elementos son del tipo siguiente:

```
struct reg1 {  
    int codigo;  
    int cantidad;  
    int  precio;};
```

- Y otro llamado "pesos.dat", cuyos elementos son del tipo siguiente:

```
struct reg2{  
    int codigo;  
    int cantidad;  
    int peso; };
```

Los elementos de ambos ficheros se encuentran ordenados ascendentemente según el campo código. Puede que haya artículos que tengan información sólo en un fichero o en los dos.

Cree un algoritmo que fusione la información de ambos ficheros en otro fichero de acceso secuencial, que se llamará "mezcla.dat", cuyos elementos estarán ordenados ascendentemente según el campo código y serán del tipo:

```
struct reg3 {  
    int codigo;  
    int cantidad;  
    int datos[2];  
    /* posición 0 : precio  
       posición 1 : peso  */  
};
```

Si un artículo tiene información en ambos ficheros, en el campo cantidad se almacenará la suma de las dos cantidades que aparecen.

```
#include <iostream>
#include <fstream>
#include <conio.h>
using namespace std;
struct reg1{
    int codigo;
    int cantidad;
    int precio;
};
struct reg2{
    int codigo;
    int cantidad;
    int peso;
};
struct reg3{
    int codigo;
    int cantidad;
    int datos[2];
};

int main() {
    reg1 r1;
    reg2 r2;
    reg3 r3;
    ifstream f1("precios.dat", ios::binary);
    if (f1){
        ifstream f2("pesos.dat", ios::binary);
        if (f2) {
            ofstream f3("mezcla.dat", ios::binary);
            if (f3) {
```

```

f1.read((char*)&r1, sizeof(reg1));
f2.read((char*)&r2, sizeof(reg2));
while (!f1.eof() && !f2.eof()){
    if (r1.codigo < r2.codigo) {
        r3.codigo      = r1.codigo;
        r3.cantidad    = r1.cantidad;
        r3.datos[0]     = r1.precio;
        r3.datos[1]     = 0;
        f1.read((char*)&r1, sizeof(reg1));
    }
    else if (r1.codigo > r2.codigo) {
        r3.codigo      = r2.codigo;
        r3.cantidad    = r2.cantidad;
        r3.datos[1]     = r2.peso;
        r3.datos[0]     = 0;
        f2.read((char*)&r2, sizeof(reg2));
    }
    else {
        r3.codigo      = r1.codigo;
        r3.cantidad    = r1.cantidad+r2.cantidad;
        r3.datos[0]     = r1.precio;
        r3.datos[1]     = r2.peso;
        f1.read((char*)&r1, sizeof(reg1));
        f2.read((char*)&r2, sizeof(reg2));
    }
    f3.write((char*)&r3, sizeof(reg3));
}
while (!f1.eof()) {
    r3.codigo      = r1.codigo;
    r3.cantidad    = r1.cantidad;
    r3.datos[0]     = r1.precio;
    r3.datos[1]     = 0;
    f1.read((char*)&r1, sizeof(reg1));
    f3.write((char*)&r3, sizeof(reg3));
}

```

```
while (!f2.eof()) {
    r3.codigo      = r2.codigo;
    r3.cantidad    = r2.cantidad;
    r3.datos[1]    = r2.peso;
    r3.datos[0]    = 0;
    f2.read((char*)&r2, sizeof(reg2));
    f3.write((char*)&r3, sizeof(reg3));
}
f1.close();
f2.close();
f3.close();
}
}
else cout <<"Error en la apertura de pesos";
}
else cout <<"Error en la apertura de precios";
getch();
return 0;
}
```